

Verifier-Guided Evaluation of LLM-Generated Network Configurations: a Paired Study with Shared Initial Candidates

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory study (CAND-2026-001) that measured whether a verifier-triggered guarded pipeline (generate $\rightarrow$ validate $\rightarrow$ repair, ≤ 1 repair) reduces deterministic-invariant violations relative to single-shot initial generation for intent $\rightarrow$ configuration NetOps tasks. Using a deterministic synthetic task generator and a deterministic network verifier, we ran 200 preregistered tasks under shared_initial_candidate semantics with the hosted model gpt-5-mini (execution/model_configuration.json records temperature = null/unset). Baseline configurations passed the verifier in 199/200 tasks (99.5%); the guarded pipeline passed 200/200 (100%). Paired absolute risk difference (guarded - baseline) = 0.005; contingency counts $n_{11}=199, n_{10}=1, n_{01}=0, n_{00}=0$; exact two-sided McNemar $p = 1.0$; paired-bootstrap 95% percentile CI = [0.00, 0.015] (10,000 resamples, seed 1729). The single guarded improvement arose from one verifier-triggered repair that corrected a multi-step ordering violation. We report preregistration, full execution accounting (201 model calls: 200 shared initial + 1 repair), paired analysis, representative diagnostic artifacts, and bounded limitations grounded in the archived execution bundle.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate high-level operator intent into device configurations for network operations (NetOps). Operational correctness for such workflows requires generated configurations to satisfy deterministic invariants (reachability, policy compliance, workflow ordering, and group-level safety). Prior prototype systems and benchmarks illustrate feasibility but leave open questions about how much deterministic verification and bounded repair alter acceptance rates when the identical initial sample is reused across comparison conditions.

This paper reports a preregistered paired experiment (CAND-2026-001) designed to isolate the marginal effect of a verifier-triggered automated repair on an identical generated candidate. Under shared_initial_candidate semantics the same single initial generation is deterministically reused by both baseline and guarded episodes; any paired difference therefore arises solely from the permitted validator-triggered repair. The primary estimand is the paired difference in per-task binary acceptance by a deterministic verifier (paired_success_rate_difference_guarded_minus_baseline). Because shared_initial_candidate semantics were enforced, this estimand measures a repair-only effect and does not capture potential benefits from independent guarded first-pass generations, constrained prompts, or multi-sample selection.

Contributions. (1) A preregistered, fully archived paired experiment executing 200 intent $\rightarrow$ configuration tasks with a deterministic verifier and a hosted LLM (gpt-5-mini) under shared_initial_candidate semantics. (2) Complete execution accounting and deterministic reconciliation showing 200 shared initial generations and 1 repair call (201 model calls total). (3) Artifact-grounded confirmatory analysis reporting baseline pass 199/200 and guarded pass 200/200 with contingency counts ($n_{11}=199, n_{10}=1, n_{01}=0, n_{00}=0$), paired risk difference 0.005, exact two-sided McNemar $p = 1.0$, and bootstrap 95% CI = [0.00, 0.015] (10,000 resamples, seed 1729). (4) A localized failure diagnosis for the single repaired pair using archived validator traces. (5) Detailed reproducibility guidance and bounded limitations tied to archived artifacts and reconciliation records.

II. RELATED WORK

We conducted a fresh literature investigation and verified records covering intent $\rightarrow$ configuration systems, generate-validate-repair patterns, and agentic-AI safety discussions relevant to NetOps. Prior intent-to-configuration prototypes and task-bench evaluations demonstrate that LLMs can synthesize device configurations from operator intents and that verification is widely advocated as a guardrail [1],[2],[3]. Generate $\rightarrow$ validate $\rightarrow$ repair has empirical support in program repair and synthesis-with-tests domains and motivates guarded NetOps pipelines, but those demonstrations are primarily in software-engineering contexts rather than artifact-backed NetOps verifier studies [4],[5],[6].

Survey and safety literature on agentic AI documents substantive failure modes (hallucination/confabulation) and argues for verifier gates and provenance; this conceptual grounding motivated our guarded design and strict oracle definition but does not supply the verifier-backed NetOps empirical outcomes reported here [7],[8]. Synthesis. The supplied, verified records indicate active prototyping and conceptual advocacy for verifier-augmented pipelines but leave an empirical gap: few reproducible paired experiments quantify the marginal repair effect when the identical initial candidate is reused. Our paired shared_initial_candidate design directly addresses that gap.

III. METHODOLOGY

Preregistration and design freeze. We preregistered study CAND-2026-001 before execution. The preregistration fixed the primary estimand, the deterministic sampling of 200 tasks drawn by `deterministic_netops_task_generator_v5` (rule-based index selection documented in the manifest), `generation_semantics = shared_initial_candidate` (one sampled initial generation per task deterministically reused), and allowed at most one automated repair call per task. The preregistration specified the confirmatory test (exact two-sided McNemar via exact binomial tail), the primary failed-call handling (`complete_pair_primary`), a sensitivity analysis treating persistent unparseables as failures, and a power rationale aiming to detect ~10–15 percentage-point paired increases at $N=200$ under mid-range baseline assumptions (~45–65%). The preregistration artifact is archived (`preregistration_sha256` in execution manifest).

Task bank, difficulty encoding, and representative statistics. Tasks were generated deterministically (`deterministic_netops_task_generator_v5`) and targeted high-difficulty workflow patterns (`very_hard` and `extreme`). Each task manifest encodes `initial_state`, `expected_final_state`, `required_sequence` (ordered command list), `allowed_touched_fields`, `workflow_policies` (e.g., at-least-one-up constraints), difficulty metadata (`changed_interface_count`, `dependency_rule_count`, `required_command_count`), and a normalized `reference_answer`. In the archived representative sample `required_command_count` spans 3–9 and `changed_interface_count` up to 3; the single baseline failure occurred on an extreme composite task with `required_command_count = 9`, consistent with concentrated failure risk on higher-complexity tasks (see `representative_task` entries).

Model configuration and sampling semantics. The declared requested model was `gpt-5-mini` (`execution/model_configuration.json`). That file records `temperature = null` (unset). We explicitly note that `temperature = null/unset` is not evidence of deterministic sampling or `temperature = 0`; nondeterministic hosted-model sampling remains possible. Provider accounting (`deterministic_reconciliation.json`) reports `stage_counts = {shared_initial_generation: 200, repair: 1}` and `hosted_model_call_event_count = 201`; `cache_miss_count = 201`, `cache_hit_count = 0` indicates fresh generation events for the shared-initial calls.

Deterministic verifier and scoring. The deterministic verifier `deterministic_netops_validator_v5` served as the oracle. It parses candidate configurations, enforces per-device allowed-field constraints, enforces `required_sequence` ordering, evaluates group invariants (e.g., `final_group_constraints`), and returns a binary `full_intent_constraint_validation` score of 1 iff all encoded invariants pass. Per-episode `validator_trace` artifacts include `validation_before` and `validation_after` states, `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, `violation_codes`, and a

boolean valid flag. These verifier traces are archived under `execution/scoring/` and `execution/responses/` and constitute the authoritative evidence for per-episode scoring.

Paired execution semantics and failure handling. Under `shared_initial_candidate` semantics baseline and guarded episodes for each pair begin from the identical sampled candidate; any paired difference therefore arises solely from the permitted verifier-triggered repair. The `missingness_plan` specified a deterministic sanitizer for minor formatting issues and a sensitivity analysis treating persistent unparseables as failures. In the archived run there were no persistent unparseables and no preregistered exclusions; `terminal_error_type` fields are null for all episodes in scoring artifacts.

Statistical analysis and exact McNemar implementation. The primary confirmatory test is the exact two-sided McNemar computed from discordant pair counts n_{10} (guarded=1, baseline=0) and n_{01} (guarded=0, baseline=1). We implement the exact two-sided McNemar via the exact binomial tail: with $n = n_{10} + n_{01}$ and $k = n_{10}$, the two-sided p-value is computed as $p = 2 * \min\{ \text{BinomCDF}(k; n, 0.5), 1 - \text{BinomCDF}(k - 1; n, 0.5) \}$ with conventional handling for $k = 0$. For our observed discordants ($n = 1, k = 1$): $\text{BinomCDF}(1; 1, 0.5) = 1$ and $1 - \text{BinomCDF}(0; 1, 0.5) = 0.5$, so $p = 2 * \min(1, 0.5) = 1.0$. Paired-bootstrap percentile confidence intervals for the absolute risk difference were computed with 10,000 resamples and `bootstrap_seed = 1729`; implementation parameters and seed are archived in `analysis/analysis_log.jsonl`.

IV. RESULTS

Execution accounting and provider audit. The archived execution manifest records start and end timestamps (UTC) showing the run elapsed ≈ 29 minutes (start 2026-09-01T16:03:05.491039+00:00, end 2026-09-01T16:32:13.298536+00:00). `Planned_episode_count = 400` and `completed_episode_count = 400`; `failed_episode_count = 0`. Provider-call accounting (`deterministic_reconciliation.json`) reports `hosted_model_call_event_count = 201` with `stage_counts = {shared_initial_generation: 200, repair: 1}`. `Cache_miss_count = 201` and `cache_hit_count = 0` indicate fresh model generations for all shared-initial calls and a single repair-stage call; these counts match the preregistered `shared_initial_candidate` semantics and the recorded single repair invocation.

Primary confirmatory outcomes. `analysis/condition_summary.csv` shows baseline successes = 199 and failures = 1 (baseline `success_rate = 0.995`) and guarded successes = 200 and failures = 0 (guarded `success_rate = 1.000`). The paired contingency table (`analysis/paired_contingency_table.csv`) records $n_{11} = 199$, $n_{10} = 1$, $n_{01} = 0$, $n_{00} = 0$. From these counts the paired absolute risk difference (guarded - baseline) = 0.005. Using the exact two-sided McNemar formulation described in Methods, with $n = n_{10} + n_{01} = 1$ and $k = n_{10} = 1$ we compute $\text{BinomCDF}(1; 1, 0.5) = 1$ and $1 - \text{BinomCDF}(0; 1, 0.5) = 0.5$; hence $p = 2 * \min(1, 0.5) = 1.0$. The paired-bootstrap percentile 95%

confidence interval for the absolute risk difference was computed with 10,000 resamples and `bootstrap_seed = 1729` (`analysis/analysis_log.jsonl`) and equals `[0.00, 0.015]`. These numeric artifacts (`analysis/condition_summary.csv`, `analysis/paired_contingency_table.csv`, `analysis/analysis_log.jsonl`) are archived and reproduce the reported confirmatory values when the `paired_binary_analysis_v1` executor is re-run on the archived `input_results_sha256`.

Single repair event and diagnostic traces. The guarded pipeline invoked exactly one automated repair. Archived validator traces for the repaired pair indicate an extreme composite task (difficulty metadata recorded in the task manifest: `required_command_count = 9`, `changed_interface_count = 3`, `difficulty.level = "extreme"`) whose shared initial candidate violated the task's `required_sequence` ordering constraint. The verifier trace for that episode documents `validation_before.violation_codes` showing the ordering breach, the `shared_initial_candidate` (the identical candidate reused by baseline and guarded), and `validation_after` with `validation_after.valid = true` once the bounded repair was applied. The repair prompt/trace and the validator's `normalized_configuration` show the localized reordering the repair enacted; after repair the scorer returned `score = 1` (`scoring_reason_code = "VALID_CONFIGURATION"`). These per-episode fields are present in the `execution/scoring/*` artifacts and constitute the archived evidence that the single observed paired improvement resulted from a verifier-triggered reordering repair that converted a sequenced-dependency failure into a valid configuration under the verifier's invariants.

Missingness, exclusions, and contamination checks. The run recorded no preregistered exclusions and no persistent unparsables; `analysis/missingness_summary.csv` reports `complete_pairs = 200` and zeros for all other missingness categories. `analysis/contamination_summary.csv` is empty (no flagged contamination). `Deterministic_reconciliation.json` indicates `pair_accounting_consistent = true` and `all_deterministic_consistency_checks_passed = true`. No episodes have `terminal_error_type` set, and provider-call audit details align with preregistration (200 shared initial generations, 1 repair). These artifacts support the statement that the confirmatory analysis included the full preregistered sample without post-hoc exclusions.

Exploratory diagnostics by difficulty metadata. Representative task manifests archived in `execution/task_manifest.jsonl` encode difficulty metadata (`changed_interface_count`, `dependency_rule_count`, `required_command_count`). In the archived representative subset `required_command_count` values span 3–9 and `changed_interface_count` values up to 3; the lone baseline failure is associated with the extreme composite difficulty pattern (highest `required_command_count` in the sample), consistent with the preregistered exploratory hypothesis that multi-device sequencing and dependency complexity concentrate residual failures. `analysis/paired_difference_figure.svg` (archived) visualizes condition rates and the bootstrap distribution used for the CI; those visual diagnostics and the per-task `validator_trace` fields (`parsed_command_count`, ob-

served_touched_field_count, violation_codes) are available for targeted reanalysis to inspect whether other high-difficulty tasks produced near-miss traces that would benefit from alternative repair strategies.

Synthesis of quantitative uncertainty. The realized baseline success rate ($199/200 = 99.5\%$) places the confirmatory test in a sparse-discordant regime: with only one discordant pair the exact McNemar p-value gives limited directional evidence ($p = 1.0$), while the paired-bootstrap CI `[0.00, 0.015]` quantifies the upper bound of plausible absolute improvement attributable to repair in this repair-only estimand. In practical terms, these results show that, for this synthetic verifier-backed task bank and the single-model run, bounded verifier-triggered repair corrected one expressible sequencing fault; they do not imply large marginal gains across broader settings without further experiments (e.g., independent guarded-generation contrasts, multi-model studies, or live dataplane emulation).

V. DISCUSSION

We expand the interpretive analysis and diagnostics here while preserving the paper's existing supported content.

Estimand interpretation and scope (explicit). Because the experiment used `shared_initial_candidate` semantics, baseline and guarded episodes for each task began from the identical sampled candidate; consequently the primary estimand intentionally measures the marginal effect of permitting an automated verifier-triggered repair on that identical candidate (a repair-only effect). This design isolates repair mechanics from any potential benefits that could arise if the guarded condition were allowed to sample independently or draw multiple candidates.

Sparse-discordant regime and inferential consequences. The observed data ($n_{11}=199$, $n_{10}=1$, $n_{01}=0$) place the analysis in a sparse-discordant regime, a setting where conventional asymptotic approximations are unreliable and exact discrete tests or resampling are appropriate. Our use of the exact binomial-tail McNemar calculation and a paired-bootstrap percentile CI (10,000 resamples, seed 1729 as archived in `analysis/analysis_log.jsonl`) is consistent with best practice for small discordant counts. Practically, the realized baseline success rate (99.5%) substantially reduced the experiment's sensitivity to detect modest repair-only effects relative to our preregistered power target (designed for mid-range baseline rates ~45–65%). The bootstrap CI `[0.00, 0.015]` quantifies that the data are compatible with effects up to 1.5 percentage points in absolute terms under our paired repair-only estimand; the preregistered design could not reliably detect larger effects because none were present.

Mechanistic diagnostic of the repaired episode. The single repair call corrected a concrete sequencing violation expressible in the task manifest's `required_sequence` field (`required_command_count = 9` in the repaired manifest). Archived `validator_trace` fields show `validation_before.violation_codes` listing the ordering breach, `parsed_command_count` equal to the presented commands, and `validation_after.valid = true` after the automated reordering

repair. These per-episode traces demonstrate: (1) the verifier’s capacity to detect ordering invariants encoded in the manifest; (2) the repair module’s ability to perform localized, semantics-preserving reorderings that the verifier accepts; and (3) that such faults concentrated in tasks with higher `required_command_count` and extreme difficulty labels in our deterministic task bank. All diagnostic fields cited above are available in `execution/scoring/task-00000X-*.json` in the archived bundle.

Construct validity of the deterministic verifier. `deterministic_netops_validator_v5` enforces a structured subset of operational invariants encoded in the task payloads (ordering, `allowed_touched_fields`, group-level up/down constraints, and explicit per-field prerequisites). This verifier therefore provides a high-fidelity oracle for manifest-encoded invariants but does not emulate dataplane behavior, vendor runtime idiosyncrasies, or external control-plane side-effects. Construct-valid conclusions should therefore be framed strictly in terms of manifest-specified, deterministically verifiable invariants; the experiment does not claim to validate dataplane reachability or vendor CLI execution semantics that lie outside the verifier’s encoding.

Design implications for future NetOps evaluations. The paired `shared_initial_candidate` design cleanly isolates repair capability; however, operational questions remain that require alternative designs: (a) independent guarded-first-pass sampling would quantify whether guarded prompting or constrained generation yields a higher baseline of valid samples before repair; (b) multi-sample selection (n-best) could reveal selection-improvement effects; and (c) multi-model comparisons would quantify model-dependent failure patterns. From an experimental-planning perspective our results highlight that preregistered power calculations must account for plausible high baseline performance—if baseline rates turn out near 100% the discordant-pair approach will often be underpowered to detect small absolute repairs-only gains.

Reproducibility, artifact usage, and recommended reanalyses. All artifacts necessary to reproduce analysis are archived: `execution/task_manifest.jsonl` (deterministic task selection and `required_sequence` fields), `execution/model_configuration.json` (declared model, noting `temperature = null/unset`), `execution/responses/` (shared-initial and per-condition responses), `execution/scoring/` (per-episode validator traces), `analysis/paired_contingency_table.csv` and `analysis/condition_summary.csv`, `deterministic_reconciliation.json` (provider accounting and `stage_counts`), and `analysis/analysis_log.jsonl` (bootstrap parameters and seed). Re-running the `paired_binary_analysis_v1` executor on the archived `input_results_sha256` reproduces the confirmatory results. For targeted reanalysis, researchers can (i) recompute bootstrap intervals under alternative resampling schemes, (ii) treat the single repaired task as an outlier to study robustness, or (iii) reclassify manifest invariants to evaluate sensitivity of verifier coverage—all without altering original model-call artifacts.

Threats to validity revisited (artifact-grounded). Internal va-

lidity: `shared_initial_candidate` semantics intentionally eliminate differences due to separate sampling, but they also prevent measuring effects of guarded prompt variants; model sampling nondeterminism remains possible because `temperature` was unset (execution/`model_configuration.json` records `temperature = null`). Construct validity: the verifier enforces only encoded invariants; omissions in the manifest (for example, dataplane-dependent invariants) are not tested. External validity: a single hosted model (`gpt-5-mini`) and a synthetic deterministic task bank limit generalizability; results do not imply identical outcomes for other models, live devices, or telemetry-driven validation. Conclusion validity: with one discordant pair the statistical evidence for a nonzero repair-only effect is weak ($p = 1.0$), though the bootstrap CI constrains plausible magnitudes.

Operational implications and cautious hypotheses. Artifact-grounded evidence demonstrates that bounded verifier-triggered repairs can fix localized ordering violations that are detectable and expressible in task manifests. From an operational standpoint this suggests a practical pattern: require explicit, machine-checkable manifest invariants for change tasks and pair LLM synthesis with deterministic validation/repair gates before human or automated deployment. We emphasize these are actionable hypotheses supported by the archived diagnostics in this controlled synthetic setting; they are not operational claims that bounded repair will resolve dataplane-level or adversarially induced failures without further evaluation.

Summary of expanded diagnostics. The archived execution bundle provides the evidentiary chain: deterministic task manifests encoding the invariants, the shared initial generated candidates, per-episode validator traces showing before/after states, provider-call accounting confirming `shared_initial_generation = 200` and `repair = 1`, and analysis artifacts (contingency table, bootstrap parameters) documenting the statistical regime. Together these artifacts ground the narrower, repair-only conclusion and illuminate concrete design decisions for future, broader evaluations (independent guarded sampling, multi-model studies, live emulation).

VI. LIMITATIONS

- Synthetic deterministic task bank and verifier: results reflect correctness under the chosen synthetic intent templates and deterministic verifier and do not guarantee similar performance on live heterogeneous production devices or telemetry.
- Single-model experiment (`gpt-5-mini`): findings do not demonstrate multi-model generality.
- `Shared_initial_candidate` semantics: baseline and guarded conditions began from the same initial generation; the study measures verifier/repair effects only and does not evaluate independent first-pass generation contrasts.
- Limited adversarial/contamination testing: the run did not include systematic adversarial retrieval or poisoned-context attacks; such threats remain an open evaluation axis.

- Oracle coverage: the deterministic verifier enforces encoded invariants but may omit runtime behaviors and device-specific semantics observable only through live execution; external validity to live deployments is therefore limited.

VII. CONCLUSION

We executed a preregistered paired experiment measuring the marginal effect of a verifier-triggered repair under `shared_initial_candidate` semantics for LLM-generated network configurations. In 200 synthetic deterministic tasks with `gpt-5-mini` and `deterministic_netops_validator_v5`, baseline acceptance was 199/200 and guarded acceptance was 200/200; a single automated repair corrected a multi-device ordering violation. Paired absolute risk difference = 0.005 with paired-bootstrap 95% CI [0.00, 0.015] (10,000 resamples, seed 1729) and exact two-sided McNemar $p = 1.0$. Artifact-grounded evidence therefore supports a constrained conclusion: verifier-guarded bounded repair can correct localized ordering faults in this synthetic verifier-backed setting. Broader operational claims require additional experiments: multi-model studies, independent-first-pass guarded semantics, adversarial contamination testing, and live-device or dataplane emulation to capture runtime semantics not modeled by the deterministic verifier.

DISCLOSURE STATEMENT

Disclosure: This run implemented preregistered study CAND-2026-001 and was executed autonomously after the autonomy lock. Declared model: `gpt-5-mini` (execution/model_configuration.json records temperature = null/unset; null/unset is not evidence of deterministic sampling or temperature = 0). Orchestration adapter: `hosted_netops_gvr_v1`. Exact initial master prompt archived at `provenance/master_prompt.txt` (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Preregistration fixed `generation_semantics = shared_initial_candidate` (one sampled initial generation per task was deterministically reused by baseline and guarded episodes) and allowed at most one automated repair call per task. Model-call accounting in the archived execution manifest reflects 200 shared initial generation calls and 1 repair call (201 model calls total). The deterministic verifier used was `deterministic_netops_validator_v5`. Humans selected the broad topic family and constrained the initial master prompt prior to the autonomy lock as permitted by the track; no human manually edited technical manuscript content after the autonomy lock. All provider traces, verifier logs, task manifests, model-configuration records, and analysis artifacts are archived in the execution bundle and indexed by the execution manifest.

REFERENCES

- [1] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
- [2] Ryan Beckett, Aarti Gupta, Ratul Mahajan, David Walker, "A General Approach to Network Configuration Verification", 2017, 10.1145/3098822.3098834
- [3] Jack Gallifant, Majid Afshar, Saleem Ameen, Yindalon Aphinyanaphongs, Shan Chen, Giovanni Cacciamani, Dina Demner-Fushman, Dmitriy Dligach, Roxana Daneshjou, Chrystinne Oliveira Fernandes, Lasse Hyldig Hansen, Adam Landman, Lisa Soleymani Lehmann, Liam G. McCoy, Timothy A. Miller, Amy C. Moreno, Nikolaj Munch, David Restrepo, Guergana Savova, Renato Umeton, Judy Wawira Gichoya, Professor Gary S. Collins, Karel G.M. Moons, Leo Anthony Celi, Danielle S. Bitterman, "The TRIPOD-LLM reporting guideline for studies using large language models", 2025, 10.1038/s41591-024-03425-5
- [4] Yunhe Li, "Test-in-the-loop LLM Repair: Verifiable Automated Program Repair on QuixBugs with a "Failing Test → Patch → Regression Test" Loop", 2024, 10.69987/jacs.2024.40206
- [5] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, Xiaolei Wang, Yifan Du, Yushuo Chen, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Peiyu Liu, Yiwen Hu, Jian-Yun Nie, Ji-Rong Wen, "A Survey of Large Language Models", 2026, 10.1007/s11704-026-60308-3
- [6] Haoran Dong, "Tool-Augmented Large Language Model Agents for Analysis: A Focused Study", 2026, 10.2139/ssrn.6647800